DevOps Complete Flow — Very Simple Real-Time
First ee **one flow** gurthu pettuko:
Requirement
↓
Task Creation
↓
Development
↓
Git Push
↓
Jenkins Build
↓
Testing
↓
Docker Image
↓
Registry
↓
Kubernetes Deployment
↓
Monitoring
↓
Alert / Incident
↓
Fix / RCA
Simple meaning:
Business ki oka requirement vachindi
Example:
“Customer ki login + payment feature kavali.”
DevOps engineer direct ga code rayadu.
Instead:
Requirement understand
→ **developers ki tasks**
→ developer code chestadu
→ Git lo push chestadu
→ Jenkins automatic ga build chestundi
→ tests run avutayi
→ Docker image create chestham
→ image registry lo store chestham
→ Kubernetes lo deploy chestham
→ monitoring lo observe chestham
→ problem vaste incident handle chestham.

1️⃣ REQUIREMENT — First em jaruguthundi?

Situation
Client cheppadu:
“Application lo Login and Payment kavali.”
Manam em chestham?
First **requirement ni understand chestham.**
Questions:

- Login ela work avvali?
- Payment evaritho integrate cheyyali?
- Security requirements enti?
- Expected users entha mandi?
- Performance requirement enti?
- Production ki eppudu kavali?

Example
Requirement:
Login
Valid username/password
↓
Login successful
Invalid username/password
↓
Error message
Enduku?
Requirement clear ga lekunte developer wrong feature build cheyyachu.
Interview Answer

“First, I understand the business requirement, clarify functional and non-functional requirements, define acceptance criteria, and make sure the requirement is clear before development starts.”

2️⃣ TASK CREATION — Requirement ni em chestham?
Requirement pedda ga untundi.
Example:
“Build Login and Payment functionality.”
Idi developer ki one task ga ivvakudadhu.
Break chestham:
Login
→ Login API
→ Database changes
→ Authentication
→ Unit tests
Payment
→ Payment API
→ Payment gateway integration
→ Database changes
→ Testing
Jira lo
Example:
DEV-101 → Login API
DEV-102 → Login DB changes
DEV-103 → Payment integration
Enduku?
Work divide chesthe:

- ఎవరు em cheyyalo clear
- progress track cheyyachu
- dependency identify cheyyachu
- duplicate work avoid cheyyachu

Interview Answer
“I break the requirement into smaller user stories and tasks, assign them to the appropriate team members, prioritize them, and track the progress through Jira or Azure Boards.”

3️⃣ DEVELOPMENT — Developer actual ga em chestadu?
Developer task teesukuntadu.
Example:
Task: Create Login API.
Developer:
Code rayadu.
Example flow:
User
↓
Login API
↓
Database
↓
Username/password verify
↓
Response
Developer code complete chestadu.
DevOps ikkada em chestadu?
DevOps engineer usually:

- development workflow support chestadu
- Git repository maintain chestadu
- CI/CD pipeline prepare chestadu
- build/test automation chestadu
- deployment automation chestadu

4️⃣ GIT — Developer code ekkadiki pampistadu?
Developer local machine lo code rasadu.
Now code ni Git repository ki push chestadu.
Example:
Developer
↓
git add
↓
git commit

↓
git push
↓
Git Repository
Enduku Git?
Because:
Who changed?
When changed?
What changed?
anni track cheyyachu.
Previous version kuda maintain avutundi.
Branch
Developer direct main lo work cheyyakunda:
main
↓
feature/login
feature branch lo work chestadu.
Complete ayyaka:
feature/login
↓
Pull Request
↓
Code Review
↓
Merge
↓
main

5️⃣ **JENKINS — Code push ayyaka em jaruguthundi?**
Important point idi.
Developer:
git push
Jenkins ki webhook notification vastundi.
Meaning:
“New code vachindi. Pipeline run cheyyi.”
Jenkins:
Git
↓
Checkout
↓
Build
↓
Unit Test
↓
Security Scan
↓
Artifact
Enduku Jenkins?
Manual ga:
Build cheyyali
Test cheyyali
Package cheyyali
Deploy cheyyali
ani every time cheyyakunda Jenkins automate chestundi.
Interview Answer
“When developers push code to Git, Jenkins is triggered automatically through a webhook. Jenkins checks out the code, builds it, runs tests and quality checks, and prepares the deployment artifact.”

6️⃣ **BUILD — Build ante actual ga em chestham?**
Suppose Java application.
Developer source code ichadu.
Source code:
Java files
Build process:
Java source code
↓
Maven
↓
Compile
↓
Test

↓
JAR
Final output:
application.jar
Idi **artifact**.
Node.js example
Node.js code
↓
npm install
↓
npm run build
↓
Production bundle
Enduku build?
Raw source code ni deployment ki suitable format lo prepare cheyyali.

7️⃣ **TEST — Build ayyaka em chestham?**
Build successful ayyaka testing.
Unit Test
One function correct ga work chestunda?
Example:
Login function
↓
valid password
↓
SUCCESS
Wrong password:
Wrong password
↓
ERROR
Integration Test
Different components kalisi work chestunnaya?
Login API
↓
Database
↓
Authentication
↓
Response
Security / Quality
Jenkins:
Unit Test
↓
Integration Test
↓
SonarQube
↓
Security Scan
Problem unte:
BUILD FAILED
Developer ki notification.
Developer fix chestadu.
Again:
Git Push
↓
Jenkins
↓
Build
↓
Test

8️⃣ **DOCKER — Successful build ayyaka em chestham?**
Suppose:
application.jar
ready undi.
Now application ni Docker image ga package chestham.
Application
↓
Dockerfile
↓
Docker Build

↓
Docker Image
Example:
myapp:1.0
Enduku Docker?
Developer machine lo work avuthundi.
But QA machine lo work avvakapothe?
Environment difference problem.
Docker lo application + required dependencies package chestham.
So:
DEV
↓
QA
↓
UAT
↓
PROD
same image use cheyyachu.

9️⃣ REGISTRY — Docker image ekkadiki velthundi?
Docker image create ayindi.
Now central storage kavali.
Example:
Docker Image
↓
AWS ECR
or
Docker Image
↓
Docker Hub
or
Docker Image
↓
Azure ACR
Enduku?
Kubernetes deployment time lo image ekkadinunchi download cheyyali?
Registry nunchi.
Flow:
Jenkins
↓
Docker Build
↓
Docker Image
↓
Push
↓
ECR

🔟 KUBERNETES — Application ni actual ga ekkada run chestham?
Now image ECR lo undi.
Example:
myapp:1.0
Kubernetes ki cheptham:
“Ee image tho application run cheyyi.”
Kubernetes:
ECR
↓
Kubernetes
↓
Pod
↓
Application Running
EKS example
AWS lo Kubernetes use chesthe:
EKS
Application:
EKS
↓
Deployment
↓
Pods

↓
Container
↓
Application
Enduku Kubernetes?
Because Kubernetes can handle:

- application deployment
- scaling
- restarting failed containers
- rolling updates
- rollback
- service discovery
- health checks

1️⃣1️⃣ DEPLOYMENT — Actual real-time flow
Complete flow:
Developer
↓
Git Push
↓
Jenkins
↓
Build
↓
Test
↓
Docker Build
↓
Docker Image
↓
ECR
↓
Kubernetes
↓
Pods
↓
Application
Idi **most important DevOps workflow**.

1️⃣2️⃣ MONITORING — Deploy ayyaka DevOps work ayipoyinda?
❌ No.
Actually deployment tarvatha important work start avutundi.
Application running undi.
Now check:
CPU
CPU = 40%
Normal.
Memory
Memory = 60%
Normal.
Error Rate
Errors = 0.5%
Normal.
Latency
Latency = 200ms
Normal.
Tools:
Prometheus
→ metrics collect
Grafana
→ dashboard
CloudWatch
→ AWS monitoring

1️⃣3️⃣ LOGS — Problem vaste ekkada chustham?
Suppose application:
HTTP 500
Users complain:
“Login not working.”

First metrics check chestham.
Then logs check chestham.
Example:
Application Log

Database connection timeout
Now problem DB connection side undani identify cheyyachu.
Tools:

- CloudWatch Logs
- ELK
- EFK
- Kibana

1️⃣4️⃣ **ALERT — DevOps engineer computer mundu 24/7 kurchuntada?**
❌ No.
Monitoring automatically alert send chestundi.
Example:
CPU > 90%
Then:
Prometheus
↓
Alert
↓
PagerDuty / Slack / Email
↓
On-call Engineer
Engineer alert receive chestadu.

1️⃣5️⃣ **INCIDENT — Alert vachindi. Next em chestham?**
Suppose:
Production API
↓
HTTP 500
↓
Many users affected
Now this is an **incident**.
Engineer:
Step 1 — Acknowledge
"I am investigating."
Step 2 — Check impact
How many users affected?
Which service?
Production or QA?
Step 3 — Check metrics
CPU?
Memory?
Latency?
Error rate?
Step 4 — Check logs
Application logs
Database logs
Kubernetes events
Step 5 — Mitigate
Immediate safe solution.
Example:
Bad pod
↓
Remove
↓
Healthy pod
↓
Traffic restored
Step 6 — Monitor
Check whether service is stable.
Step 7 — RCA
After service restoration:
Why did this happen?

1️⃣6️⃣ **RCA — Actual root cause ela find chestham?**

Example:
Website Down
↓
Pods unhealthy
↓
Memory high
↓
Application memory leak
↓
New release
↓
Bug in new release
Root cause:
Application memory leak introduced in the new release.
Then preventive action:
Fix code
↓
Add test
↓
Deploy
↓
Monitor

🔥 **NOW FULL REAL-TIME EXAMPLE**
Interviewer:
"Explain your DevOps workflow."
You can say:
"First, we understand the business requirements and break them into smaller tasks. Developers implement the requirements and push the code to Git. A Jenkins pipeline is triggered automatically through a webhook. Jenkins checks out the code, builds the application, runs unit and integration tests, performs code-quality and security checks, and creates the deployment artifact. We then build a Docker image and push it to a container registry such as ECR. The image is deployed to Kubernetes using manifests or Helm. After deployment, we monitor application metrics, logs, and alerts using tools such as CloudWatch, Prometheus, and Grafana. If a production issue occurs, we investigate the impact, check metrics and logs, mitigate the issue, restore the service, and then perform RCA and preventive actions."

🧠 **ONE-LINE MEMORY TRICK**
Idi matram **daily repeat chesi gurthu pettuko**:
Requirement → Task → Code → Git → Jenkins → Build → Test → Docker → Registry → Kubernetes → Monitor → Alert → Incident → Fix → RCA
Inka simple ga:
"Code ni teesukoni → test chesi → package chesi → deploy chesi → monitor chesi → problem vaste fix cheyyadam."
Idi **DevOps workflow yokka core idea**.